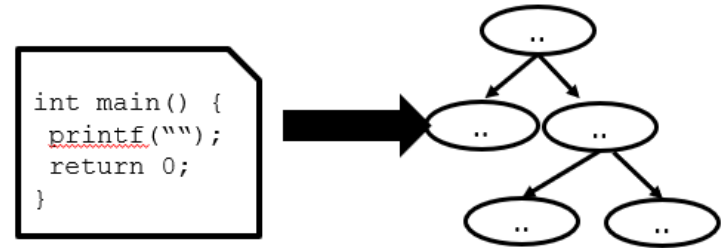


CSE110A: Compilers



Topic:

Bottom Up Parsing

Lecturer: Mrcelo Siero

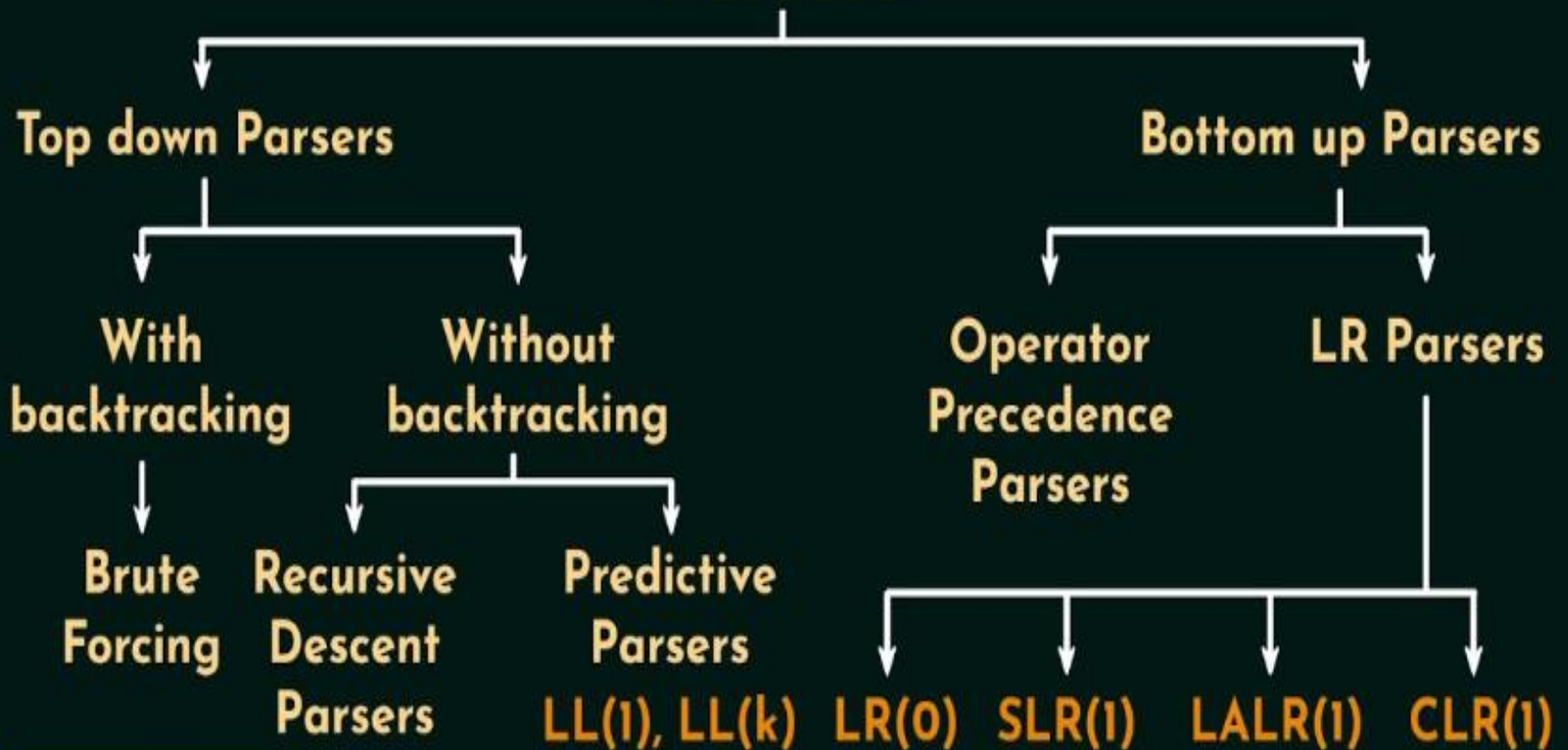
Accreditation:

Most slides taken directly from Stanfords: CS143 Lecture 8
slide design by: Prof. Alex Aiken with some modifications by
Marcelo Siero

Source: <https://web.stanford.edu/class/cs143/lectures/lecture08.pdf>

Introduction to Parsers

Parsers



Source: <https://www.youtube.com/watch?v=OIKL6wFjFOo>

Some Definitions

Context Free Grammar: Formally, a context-free grammar G is a quadruple (T, NT, S, P) where: T is a set of terminals, NT is a set of non-terminals, S is a Start symbol, and P a set of Productions all for language $L(G)$.

Ambiguity: A grammar G is *ambiguous* if some sentence in $L(G)$ has more than one rightmost (or leftmost) derivation.

Sentential Form: a string of symbols that occurs as one step in a valid derivation

1	$Expr$	$\rightarrow$	$(Expr)$
2		$ $	$Expr Op name$
3		$ $	$name$
4	Op	$\rightarrow$	$+$
5		$ $	$-$
6		$ $	$\times$
7		$ $	$\div$

Derivation: a sequence of rewriting steps that begins with the grammar's start symbol and ends with a sentence in the language

BNF grammar for Expressions

1	$Statement$	$\rightarrow$	$if Expr then Statement else Statement$
2		$ $	$if Expr then Statement$
3		$ $	$Assignment$
4		$ $	$... other statements ...$

Sentence: a string of symbols that can be derived from the rules of a grammar, i.e. a sentential form without non-terminals.

Classical ambiguous grammar from Algol 60

Leftmost and Rightmost Derivations

1	$Expr \rightarrow$	$(Expr)$
2		$Expr Op \text{ name}$
3		name
4	$Op \rightarrow$	$+$
5		$-$
6		$\times$
7		$\div$

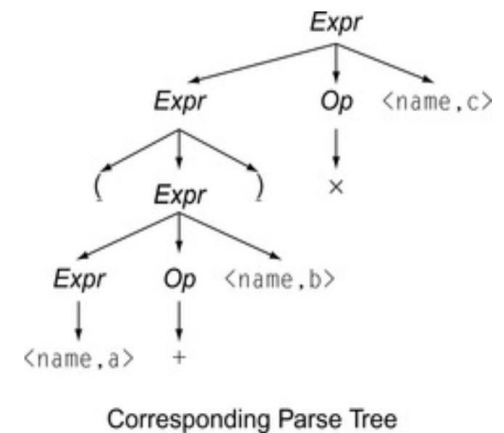
Note that leftmost derivations tend to be right associative
Rightmost derivations left associative.

Rule	Sentential Form
	$Expr$
2	$Expr Op \text{ name}$
1	$(Expr) Op \text{ name}$
2	$(Expr Op \text{ name}) Op \text{ name}$
3	$(\text{name} Op \text{ name}) Op \text{ name}$
4	$(\text{name} + \text{name}) Op \text{ name}$
6	$(\text{name} + \text{name}) \times \text{name}$

Leftmost Derivation of $(a + b) \times c$

Rule	Sentential Form
	$Expr$
2	$Expr Op \text{ name}$
6	$Expr \times \text{name}$
1	$(Expr) \times \text{name}$
2	$(Expr Op \text{ name}) \times \text{name}$
4	$(Expr + \text{name}) \times \text{name}$
3	$(\text{name} + \text{name}) \times \text{name}$

Rightmost Derivation of $(a + b) \times c$



Bottom-Up Parsing

- Bottom-up parsing is more general than top-down parsing
 - And just as efficient
 - Builds on ideas in top-down parsing
- Bottom-up is a very popular method
- Concepts: State Table Algorithms for Advanced
- Course in compilers.

Bottom-Up Parsing

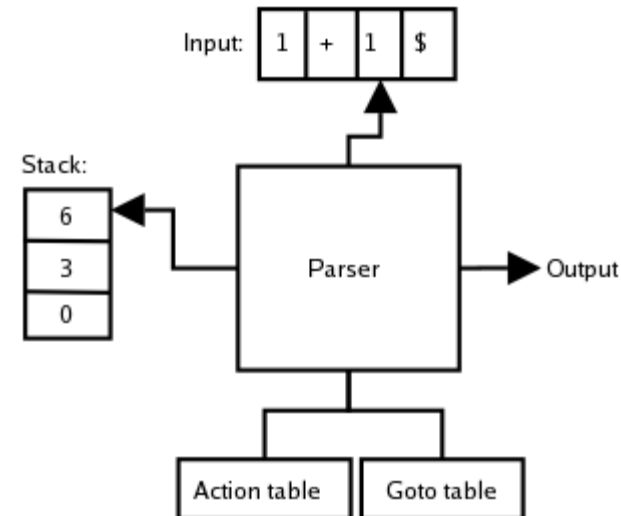
example of LR parsing

- (1) $E \rightarrow E * B$
 (2) $E \rightarrow E + B$ T: *, +, 0, 1
 (3) $E \rightarrow B$ NT: E, B
 (4) $B \rightarrow 0$ Start: E
 (5) $B \rightarrow 1$

state	action					goto	
	*	+	0	1	\$	E	B
0			s1	s2		3	4
1	r4	r4	r4	r4	r4		
2	r5	r5	r5	r5	r5		
3	s5	s6			acc		
4	r3	r3	r3	r3	r3		
5			s1	s2			7
6			s1	s2			8
7	r1	r1	r1	r1	r1		
8	r2	r2	r2	r2	r2		

LR parsing maintains a stack of states.

There is a 1-to-1 correspondence:
each grammar symbol shifted/reduced to
exactly one state pushed on the stack



Actions:

Shift: **sn** , push state **n**, eat input token

Reduce: **rm**, reduce with rule **m**

Accept: **acc**, indicates input string accepted. 6

Bottom-Up Parsing (Pseudo Code from EAC Ch3.4)

```
push $;
push start_state: s0;
word = NextWord();

while (true)do:
    state = top_of_stack
    if (Action[state.word] = "reduce A::=b"):
        pop 2 * |b| symbols;
        state = top_of_stack
        push A;
        push Goto[state, A];
    elif (Action[state.word] = "shift si"):
        push word;
        push si;
        word = NextWord();
    elif (Action[state.word] = "accept"):
        break
    else:
        Fail();
report_success
```

S	: Expr
1. Expr	: Expr + Term
2.	Expr - Term
3.	Term
4. Term	: Term * Factor
5.	term / Factor
6.	Factor
7. Factor	: (Expr)
8.	num
9.	name

Bottom-Up Parsing (Pseudo Code from EAC Ch3.4)

re-written for clarity

```
# Initialize stack with start state and end marker
stack = [(s0, None)] # None means no symbol yet
lookahead = NextWord() # first input symbol

while True:
    # Get the current parser state from top of stack
    state = stack[-1][0] # top tuple: (state, symbol)
    # Look up action in the parsing table using (state, lookahead)
    action = Action[state, lookahead]
    if action is "reduce A ->  $\beta$ ":
        # Pop 2 *  $|\beta|$  elements (state, symbol pairs) for RHS
        for i in range(len( $\beta$ )):
            stack.pop()
        # Top of stack is now the state before  $\beta$ 
        prev_state = stack[-1][0]
        # Push LHS nonterminal A
        stack.append((Goto[prev_state, A], A))
    elif action is "shift s_i":
        # Push the terminal and new state
        stack.append((s_i, lookahead))
        # Advance input
        lookahead = NextWord()
    elif action is "accept":
        break
    else:
        Fail() # syntax error
```

S	: Expr
1. Expr	: Expr + Term
2.	Expr - Term
3.	Term
4. Term	: Term * Factor
5.	term / Factor
6.	Factor
7. Factor	: (Expr)
8.	num
9.	name

An Introductory Example

- Bottom-up parsers don't need left-factored grammars
- Revert to the “natural” grammar for our example:

$E \rightarrow T + E$

| T

$T \rightarrow \text{int} * T$

| int

| (E)

- Consider the string: $\text{int} * \text{int} + \text{int}$

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

The Idea

Bottom-up parsing reduces a string to the start symbol by inverting productions:

int * int + int

$T \rightarrow \text{int}$

int * T + int

$T \rightarrow \text{int} * T$

T + int

$T \rightarrow \text{int}$

T + T

$E \rightarrow T$

T + E

$E \rightarrow T + E$

E

$$E \rightarrow T + E \mid T$$

$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

Observation

- Read the productions in reverse (bottom to top)
- This is a reverse rightmost derivation!

int * int + int

$T \rightarrow \text{int}$

int * T + int

$T \rightarrow \text{int} * T$

T + int

$T \rightarrow \text{int}$

T + T

$E \rightarrow T$

T + E

$E \rightarrow T + E$

E

Important Fact #1

Important Fact #1 about bottom-up parsing:

A bottom-up parser traces a **rightmost derivation in reverse**, i.e. from a Sentence to Start symbol.

A Bottom-up Parse

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

int * int + int

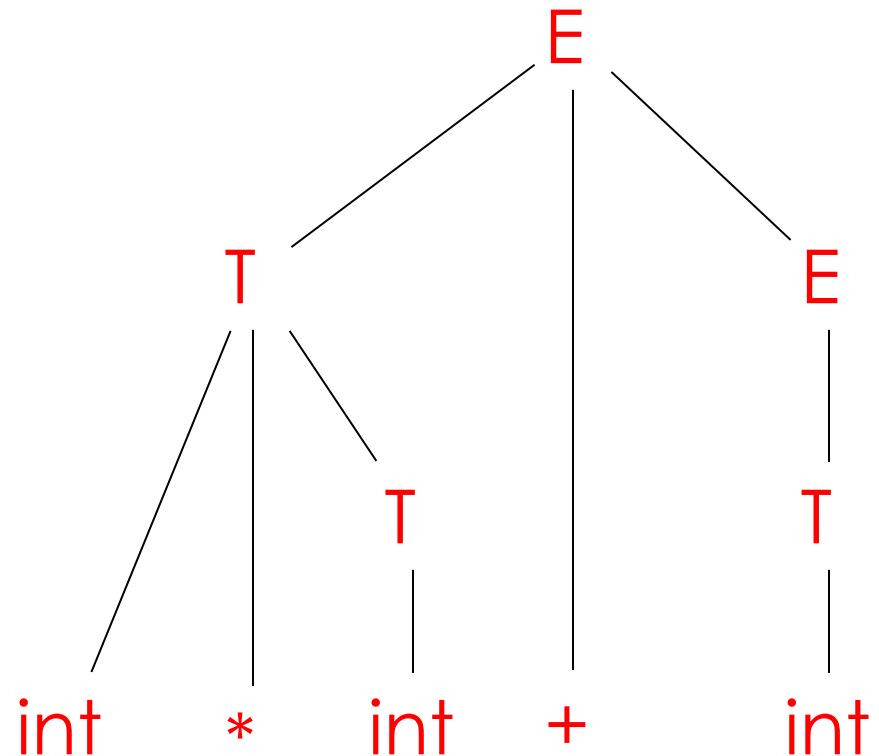
int * T + int

T + int

T + T

T + E

E



A Bottom-up Parse in Detail (1)

$$\begin{aligned} E &\rightarrow T + E \mid T \\ T &\rightarrow \text{int} * T \mid \text{int} \mid (E) \end{aligned}$$

int * int + int

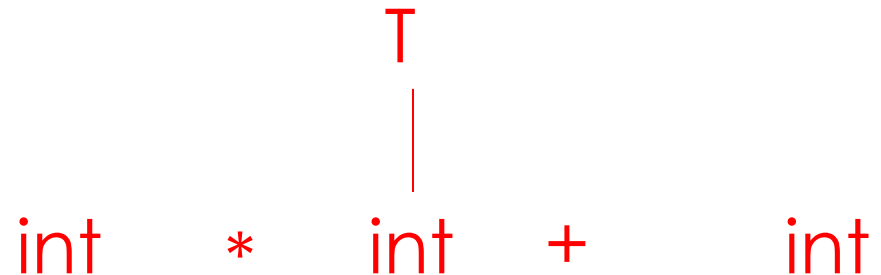
int * int + int

A Bottom-up Parse in Detail (2)

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

int * int + int

int * T + int



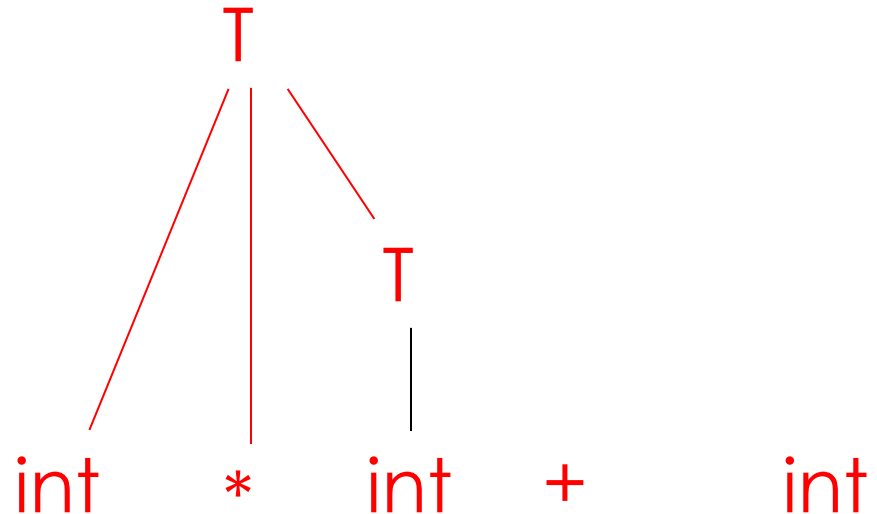
A Bottom-up Parse in Detail (3)

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

int * int + int

int * T + int

T + int



A Bottom-up Parse in Detail (4)

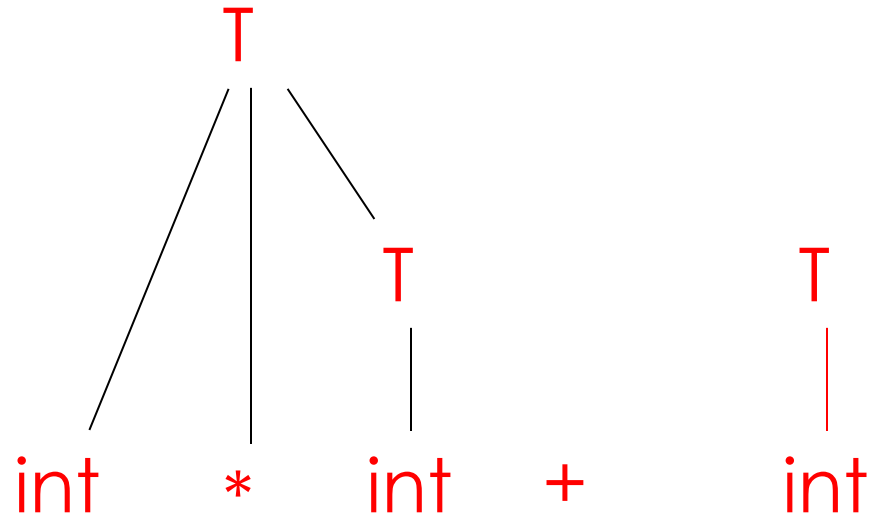
$E \rightarrow T + E \mid T$
 $T \rightarrow \text{int} * T \mid \text{int} \mid (E)$

int * int + int

int * T + int

T + int

T + T



A Bottom-up Parse in Detail (5)

$E \rightarrow T + E \mid T$
 $T \rightarrow \text{int} * T \mid \text{int} \mid (E)$

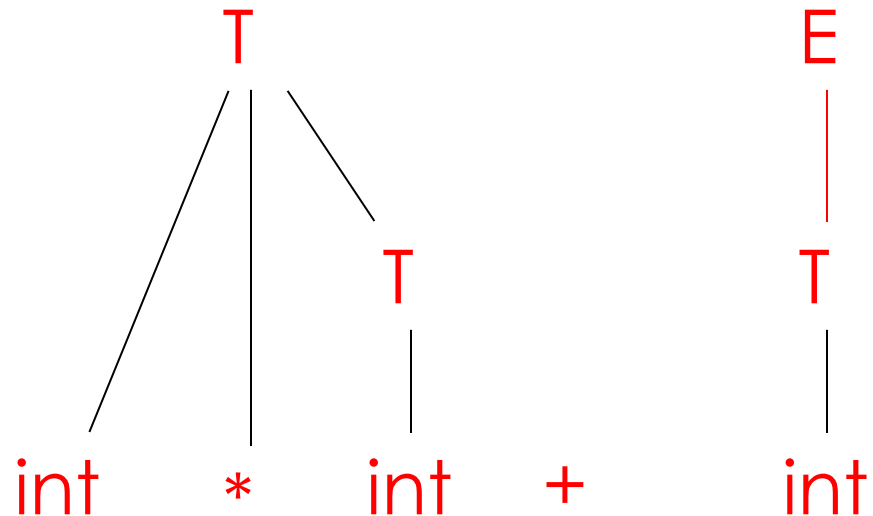
int * int + int

int * T + int

T + int

T + T

T + E



A Bottom-up Parse in Detail (6)

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

int * int + int

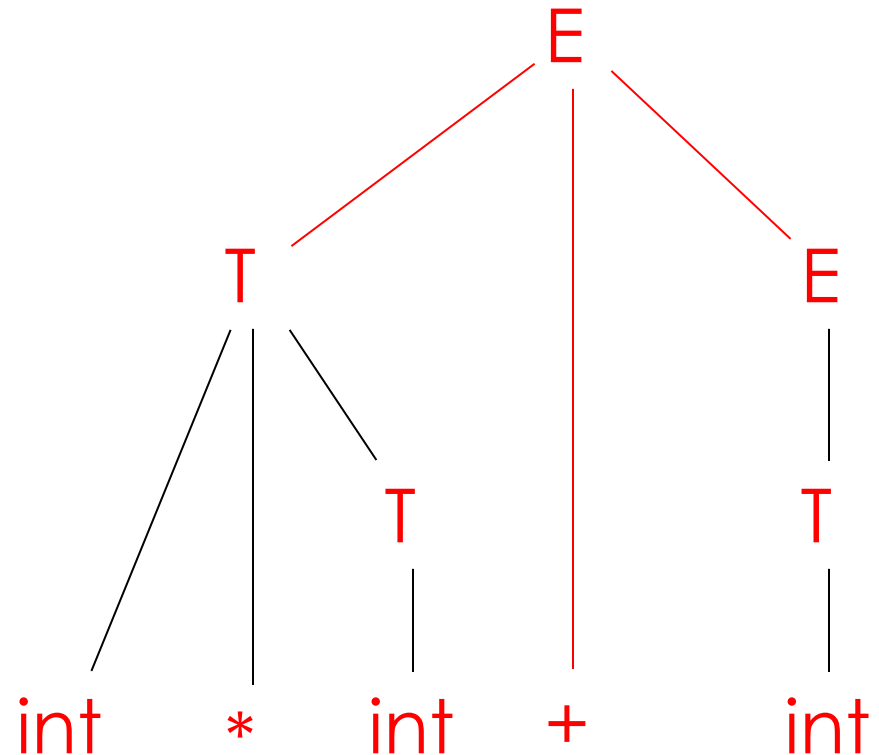
int * T + int

T + int

T + T

T + E

E



Where Do Reductions Happen?

Important Fact #1 has an interesting consequence:

- Let $\alpha\beta\omega$ be a step of a bottom-up parse
- Assume the next reduction is by $X \rightarrow \beta$
- Then ω is a string of terminals

Why? Because $\alpha X\omega \rightarrow \alpha\beta\omega$ is a step in a right-most derivation

Notation

- Idea: Split string into two substrings
 - Right substring is as yet unexamined by parsing (a string of terminals)
 - Left substring of sentential form has both terminals and non-terminals
- The dividing point is marked by a |
 - The | is not part of the string
- Initially, all input is unexamined | $x_1x_2 \dots x_n$

Shift-Reduce Parsing

Bottom-up parsing uses only two kinds of actions:

Shift (Push onto a Stack)

Reduce (Apply a production rule)

Shift

- Shift: Move | one place to the right
 - Shifts a terminal to the leftstring

ABC|xyz $\Rightarrow$ ABCx|yz

Reduce

- Apply an inverse production at the right end of the left string
 - If $A \rightarrow xy$ is a production, then

$$Cbxy|ijk \Rightarrow CbA|ijk$$

The Example with Reductions Only

$$\begin{aligned} E &\rightarrow T + E \mid T \\ T &\rightarrow \text{int} * T \mid \text{int} \mid (E) \end{aligned}$$

int * int | + int

reduce $T \rightarrow \text{int}$

int * T | + int

reduce $T \rightarrow \text{int} * T$

T + int |

reduce $T \rightarrow \text{int}$

T + T |

reduce $E \rightarrow T$

T + E |

reduce $E \rightarrow T + E$

E |

The Example with Shift-Reduce Parsing

$E \rightarrow T + E \mid T$
 $T \rightarrow \text{int} * T \mid \text{int} \mid (E)$

int * int + int	shift
int * int + int	shift
int * int + int	shift
int * int + int	reduce $T \rightarrow \text{int}$
int * T + int	reduce $T \rightarrow \text{int} * T$
T + int	shift
T + int	shift
T + int	reduce $T \rightarrow \text{int}$
T + T	reduce $E \rightarrow T$
T + E	reduce $E \rightarrow T + E$
E	

A Shift-Reduce Parse in Detail (1)

$E \rightarrow T + E \mid T$
 $T \rightarrow \text{int} * T \mid \text{int} \mid (E)$

| int * int + int

int * int + int
↑

A Shift-Reduce Parse in Detail (2)

$E \rightarrow T + E \mid T$
 $T \rightarrow \text{int} * T \mid \text{int} \mid (E)$

| int * int + int

int | * int + int

int * int + int
↑

A Shift-Reduce Parse in Detail (3)

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

| int * int + int

int | * int + int

int * | int + int

int * int + int

↑

A Shift-Reduce Parse in Detail (4)

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

| int * int + int

int | * int + int

int * | int + int

int * int | + int

int * int + int

↑

A Shift-Reduce Parse in Detail (5)

$E \rightarrow T + E \mid T$
 $T \rightarrow \text{int} * T \mid \text{int} \mid (E)$

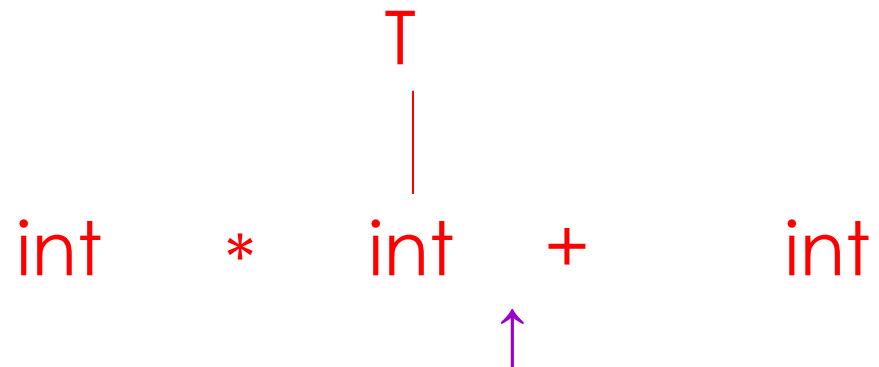
| int * int + int

int | * int + int

int * | int + int

int * int | + int

int * T | + int



$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

A Shift-Reduce Parse in Detail (6)

| int * int + int

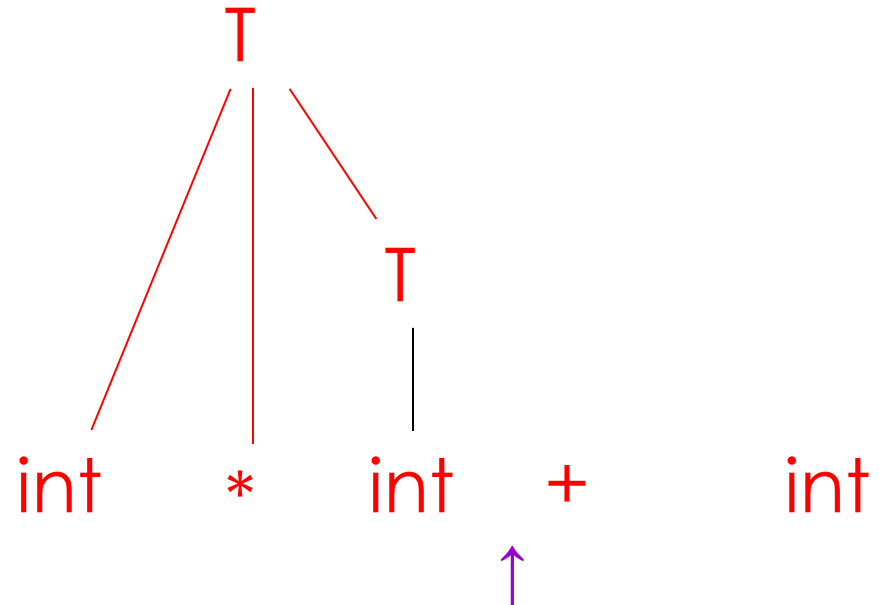
int | * int + int

int * | int + int

int * int | + int

int * T | + int

T | + int



$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

A Shift-Reduce Parse in Detail (7)

| int * int + int

int | * int + int

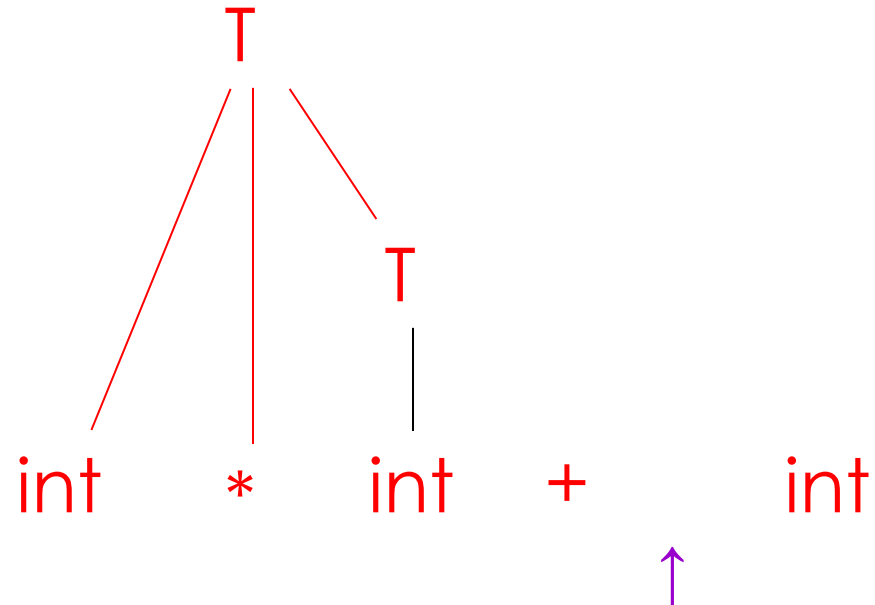
int * | int + int

int * int | + int

int * T | + int

T | + int

T + | int



A Shift-Reduce Parse in Detail (8)

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

| int * int + int

int | * int + int

int * | int + int

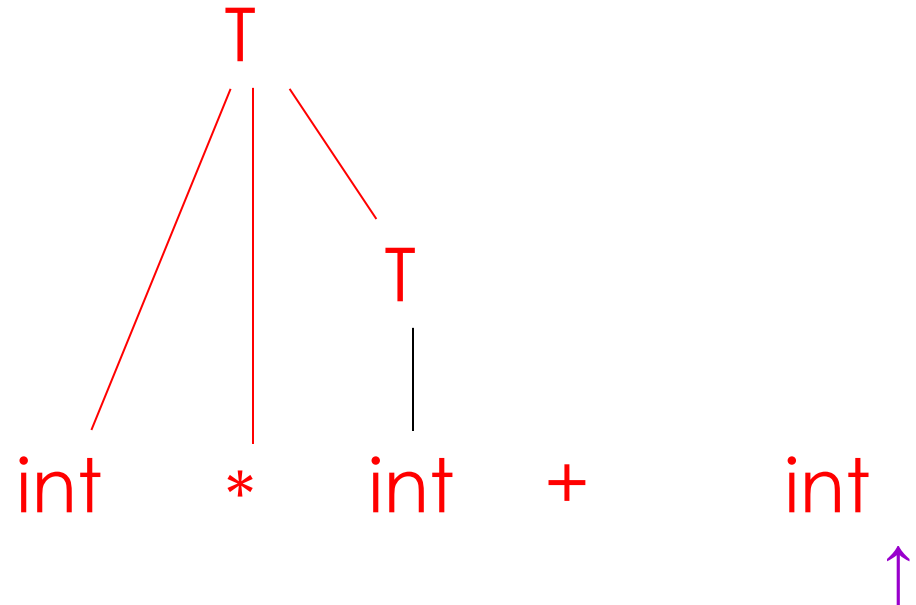
int * int | + int

int * T | + int

T | + int

T + | int

T + int |



A Shift-Reduce Parse in Detail (9)

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

| int * int + int

int | * int + int

int * | int + int

int * int | + int

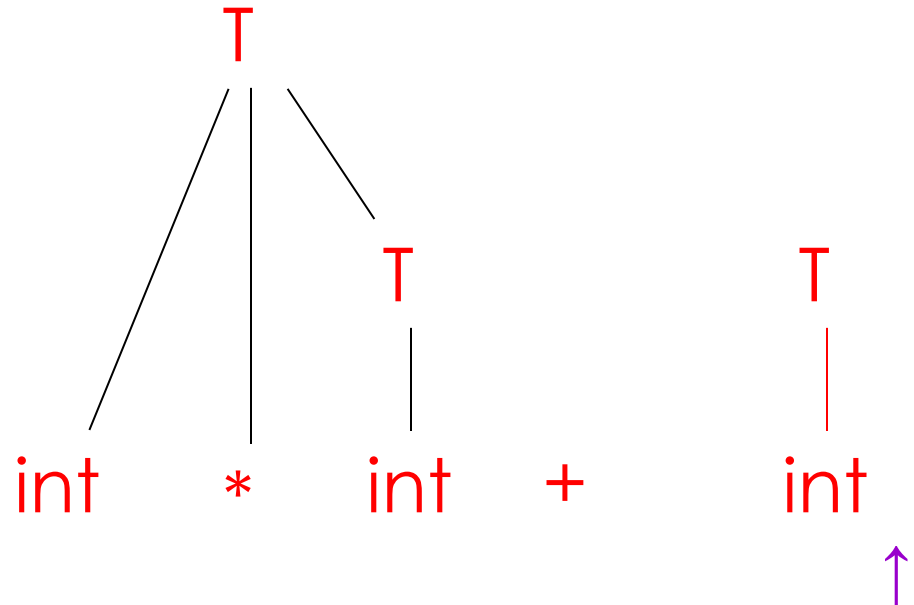
int * T | + int

T | + int

T + | int

T + int |

T + T |



A Shift-Reduce Parse in Detail (10)

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

| int * int + int

int | * int + int

int * | int + int

int * int | + int

int * T | + int

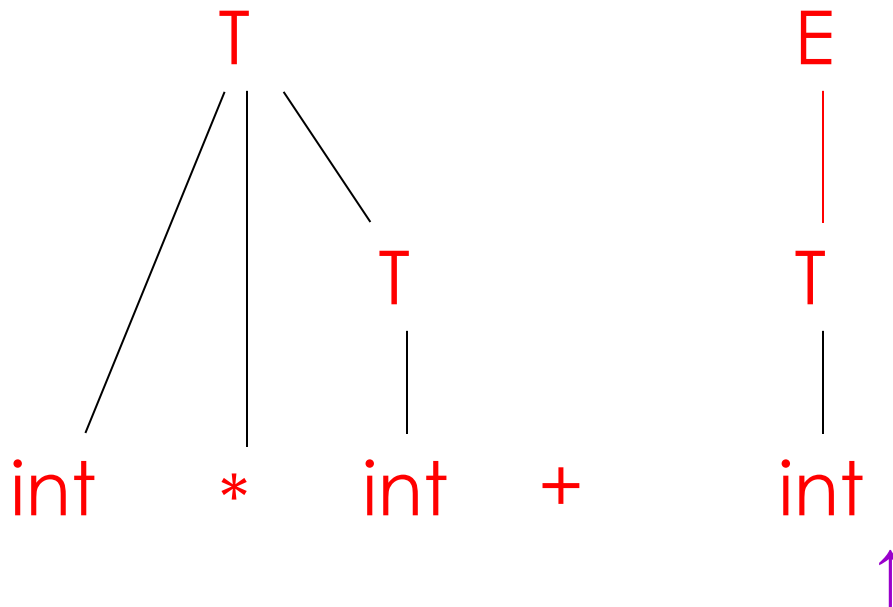
T | + int

T + | int

T + int |

T + T |

T + E |



A Shift-Reduce Parse in Detail (11)

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

| int * int + int

int | * int + int

int * | int + int

int * int | + int

int * T | + int

T | + int

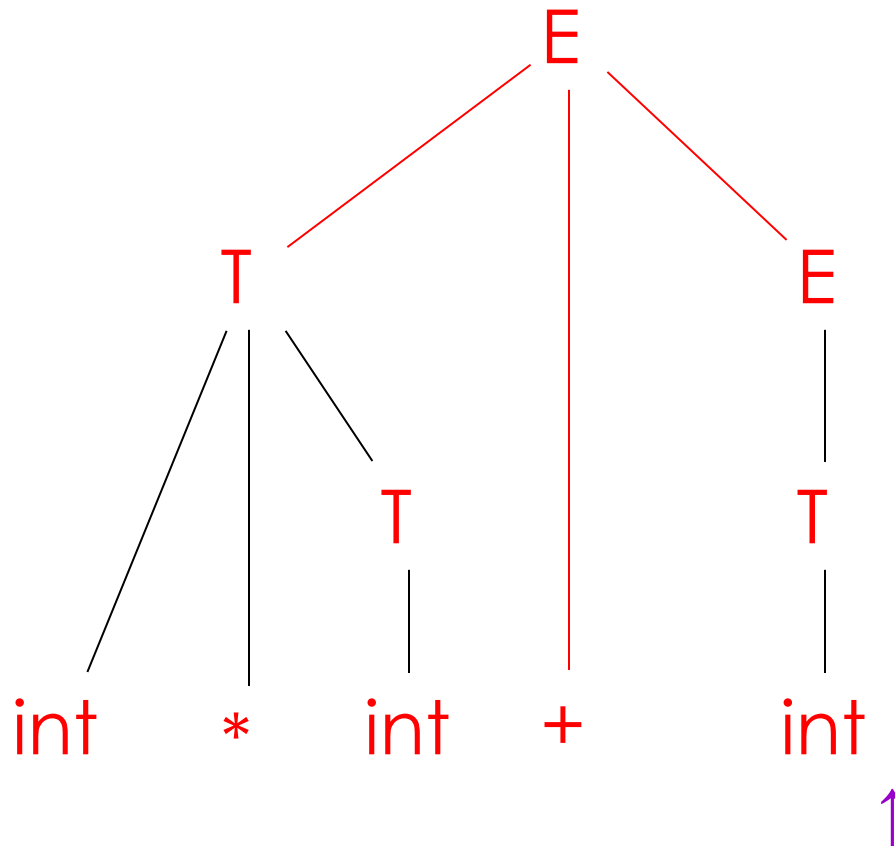
T + | int

T + int |

T + T |

T + E |

E |



The Stack

- Left string can be implemented by a stack
 - Top of the stack is the |
- **Shift pushes a terminal on the stack**
- **Reduce pops 0 or more symbols off of the stack (production RHS) and pushes a non-terminal on the stack (production LHS)**

Conflicts

- In a given state, more than one action (shift or reduce) may lead to a valid parse
- If it is legal to shift or reduce, there is a **shift-reduce** conflict
- If it is legal to reduce by two different productions, there is a **reduce-reduce** conflict
- When using a parser generator you may well see conflicts, that will have to be disambiguated.

Key Issue

- How do we decide when to shift or reduce?
- Example grammar:
$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$
- Consider step $\text{int} \mid * \text{int} + \text{int}$
 - We could reduce by $T \rightarrow \text{int}$ giving $T \mid * \text{int} + \text{int}$
 - A fatal mistake!
 - No way to reduce to the start symbol E

Definition: Handles

- Intuition: Want to reduce only if the result can still be reduced to the start symbol S

- Assume a rightmost derivation

$$S \rightarrow^* \alpha X \omega \rightarrow \alpha \beta \omega$$

- Then $X \rightarrow \beta$ in the position after α is a **handle** of $\alpha \beta \omega$
- Can and must reduce at handles.

Handles (Cont.)

- Handles formalize the intuition
 - **A handle is a string that can be reduced and also allows further reductions back to the start symbol (using a particular production at a specific spot)**
- We only want to reduce at handles
- Note: We have said what a handle is, not how to find handles

Important Fact #2

Important Fact #2 about bottom-up parsing:

In shift-reduce parsing, **handles** appear only **at the top of the stack**, never inside the stack

Why?

- Informal induction on # of reduce moves:
- True initially, stack is empty
- Immediately after reducing a handle
 - right-most non-terminal on top of the stack
 - next handle must be to right of right-most non-terminal, because this is a right-most derivation
 - Sequence of shift moves reaches next handle

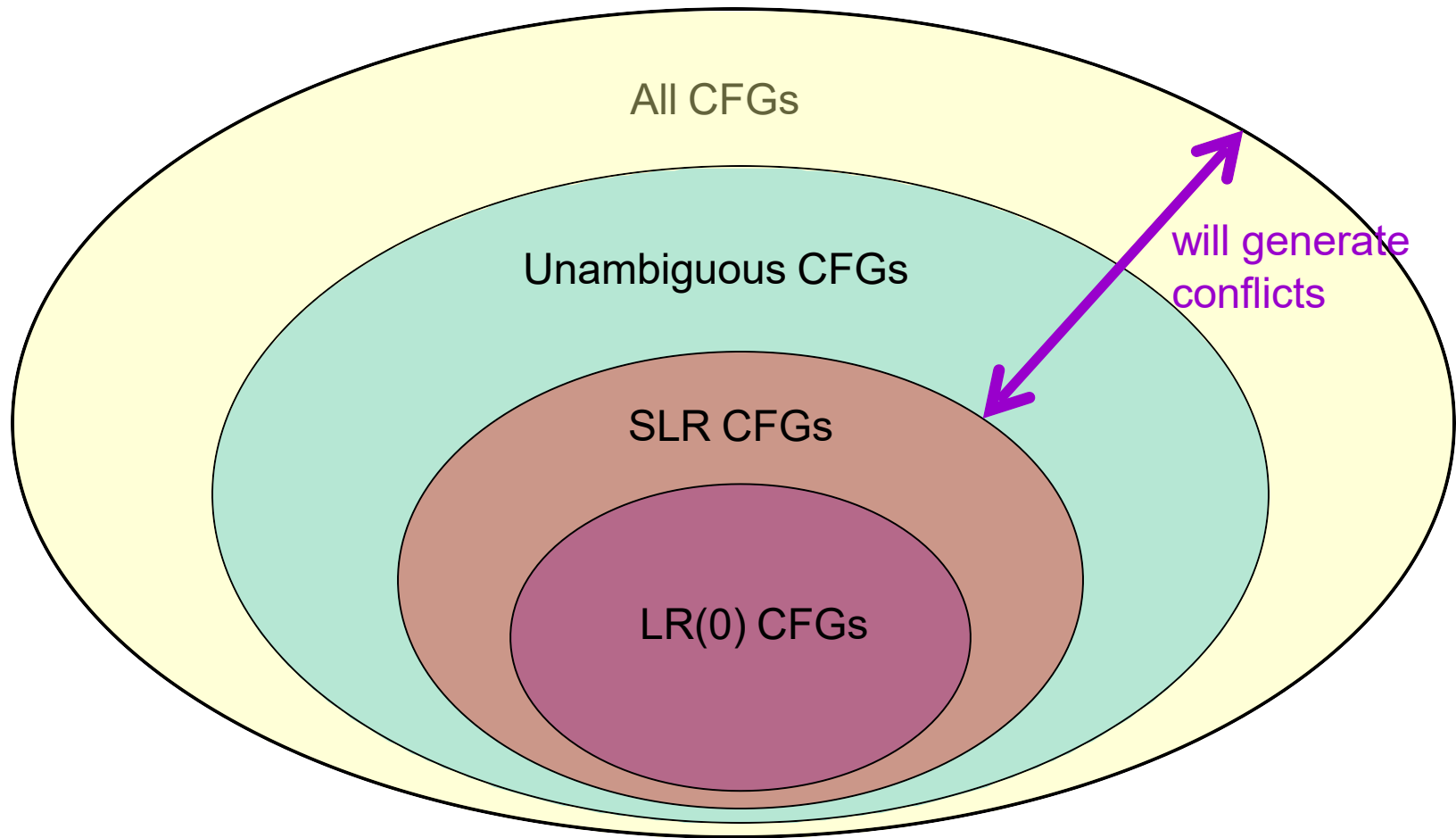
Summary of Handles

- In shift-reduce parsing, handles always appear at the top of the stack
- **Parsing decisions** are made by scanning from **left to right**, but we only reduce **rightmost** valid production (handle). Parser only needs to examine the stack up to the handle at the top; it does not need to look further to the left.
- Bottom-up parsing algorithms are based on recognizing handles

Recognizing Handles

- There are grammars with no known efficient algorithms to recognize handles
- Some CFGs, use proofs to guarantee either finding the handle, or determining its ambiguous conflict. These are known as:
SLR(1), LALR(1), Canonical LR(1),
 - For the heuristics we use here, these are the SLR grammars
 - Other heuristics work for other grammars

Grammars



Categories of Bottom Up Grammars

Parser	Lookahead	Table Size	ϵ Allowed	Deterministic	Notes
Simple	0	Small	✗	Sort of	Precedence matrix only
Operator	0	Small	✗	Sort of	Binary ops only
LR(0)	0	Small	✓	✓	Rarely sufficient
SLR(1)	1	Small	✓	✓	Uses FOLLOW sets
LALR(1)	1	Medium	✓	✓	Merged lookaheads
Canonical LR1	1	Large	✓	✓	Most precise

Canonical LR(1) is “most precise” because it uses full lookahead to make reductions, handling all grammars that an LR(1) parser can handle without conflicts, unlike SLR(1) or LALR(1).